大模型数值精度全解：FP32、BF16、FP16、FP8、INT8、INT4 与 FP4

> **原始问题：**详细讲解不同精度：bf32, 16, 8等等，有什么区别，作用在哪里，如何影响训练的性能和显存等指标

提问：2026/08/02 21:09　更新：2026/08/02 21:16　重要度：5/5

大模型数值精度全解：FP32、TF32、BF16、FP16、FP8、INT8、INT4 与 FP4

先纠正一个术语：`BF32` 通常不是你真正想问的格式

在主流 PyTorch、CUDA、NVIDIA Tensor Core 与大模型训练语境中，常见的是：

- `FP32`：32-bit IEEE single precision；
- `TF32`：TensorFloat-32，NVIDIA 的矩阵计算模式；
- `BF16`：bfloat16，16-bit Brain Floating Point；
- `FP16`：IEEE half precision；
- `FP8`：常见 E4M3、E5M2；
- `INT8 / INT4`：整数定点量化；
- `FP4`：例如 Blackwell 上的 NVFP4 E2M1。

`BF32` **不是当前 PyTorch/CUDA 大模型训练栈里的统一主流标准名称。**有些非权威资料会自行使用这个词，但不能把它当作像 BF16、FP32 那样稳定且通用的格式。结合你的表达“bf32、16、8”，本文按你想系统比较 **FP32、16-bit、8-bit 及更低精度** 来讲。

一句话结论

1. **位宽越低，单个元素占用的显存、HBM 流量和通信 payload 越小，支持该格式的 Tensor Core 吞吐通常越高。**
2. **位宽越低，能够表示的数值范围或相邻数密度越差，更容易发生溢出、下溢、舍入误差和累加误差。**
3. **大模型训练几乎从来不是“全模型只用一种精度”，而是混合精度：**

- GEMM/卷积等大计算使用 BF16、FP16、FP8；
- 累加、Softmax、归一化、损失和部分归约使用 FP32 或其他较高精度；
- Adam 的一阶/二阶状态通常仍保留 FP32；
- FP16 常配 Dynamic Loss Scaling；
- FP8/FP4 还需要逐张量或逐块 Scaling。

1. **BF16 是现代大模型训练的通用稳健默认值；FP16 范围小、通常需要 Loss Scaling；FP8 是更激进的训练优化；INT8/INT4 主要用于推理量化；FP4 训练属于新硬件与专用 Recipe 驱动的前沿方案。**
2. **“理论 FLOPS 翻倍”不等于 Step Time 减半。**端到端收益取决于 Tensor Core 覆盖率、Memory-Bound 比例、非 GEMM 算子、通信、Cast/Scale 开销、Shape 对齐和数值回退。

一、浮点数究竟如何表示一个数

一个规格化二进制浮点数可以写成：

```
value = (−1)^sign × 2^exponent × significand
```

格式通常分成三部分：

```
Sign        符号位：决定正负
Exponent    指数位：决定数量级范围
Mantissa    尾数/小数位：决定同一数量级内能刻多细
```

直觉：

- **指数位多**：能表示特别大和特别小的数，不容易 Overflow/Underflow；
- **尾数位多**：相邻可表示数更密，舍入误差更小；

- **总位宽少**：必须在“范围”和“精细程度”之间做更激进的取舍。



1.1 范围与精度不是同一件事

假设都在 `1.0` 附近：

```
FP32 下一个可表示数约为：1 + 2^-23
FP16 下一个可表示数约为：1 + 2^-10
BF16 下一个可表示数约为：1 + 2^-7
```

所以：

```
1.0 + 0.0001
```

在 FP32 中可以产生可见变化，但在 FP16、BF16 中可能仍被舍入回 `1.0`。这就是“小更新被大数吃掉”。

然而 BF16 虽然比 FP16 尾数更少，却拥有与 FP32 一样宽的 8-bit 指数，因此能覆盖接近 FP32 的数量级范围。

二、常见精度格式总表

下表中的动态范围和 `epsilon near 1` 用于建立量级直觉。具体硬件可能对 Subnormal、NaN/Inf、累加方式和舍入模式有额外行为。

格式	总位数	Sign / Exp / Mantissa	每元素字节	最大有限值（约）	1附近相对间隔（约）	核心用途
FP64	64	1 / 11 / 52	8 B	`1.8e308`	`2.22e-16`	科学计算、数值敏感求解
FP32	32	1 / 8 / 23	4 B	`3.4e38`	`1.19e-7`	稳定基线、累加、优化器状态
TF32	计算有效位约19	1 / 8 / 10	输入/输出通常仍以FP32存储	接近FP32	`9.77e-4` 量级	Ampere及以后加速FP32 GEMM/Conv
BF16	16	1 / 8 / 7	2 B	`3.39e38`	`7.81e-3`	现代大模型训练默认低精度
FP16	16	1 / 5 / 10	2 B	`65504`	`9.77e-4`	混合精度训练、旧硬件兼容
FP8 E4M3	8	1 / 4 / 3	1 B	`448`	很粗	常用于前向权重/激活
FP8 E5M2	8	1 / 5 / 2	1 B	`57344`	更粗	常用于范围更大的梯度
INT8	8	整数，无指数	1 B	有符号通常 `-128..127`	固定步长	推理权重/激活量化、部分通信
INT4	4	整数，无指数	0.5 B	有符号量化范围依方案	固定步长	Weight-Only推理量化
FP4 E2M1	4	1 / 2 / 1	0.5 B	原始值幅度约到 `6`	极粗	NVFP4等块缩放训练/推理

两个重要说明：

1. **TF32 不是通常意义上把模型权重存成 19-bit 的 dtype。**它主要是 Tensor Core 执行 FP32 GEMM/Conv 时对输入有效尾数做截短/舍入，输出与累加仍走 FP32 语义或较高精度路径。
2. **FP8/FP4 的原始范围不能脱离 Scaling 来看。**真实训练通过 Scale 把张量当前范围映射进很窄的低精度格式。

三、逐个讲清每种精度

3.1 FP64：大模型训练通常不需要，但数值科学需要

FP64 有约 16 位十进制有效数字，范围也极大。

适合：

- 数值线性代数；
- 高条件数问题；
- 分子动力学、计算流体、科学仿真；
- 需要严格误差控制的统计或优化算法。

不适合常规 LLM 训练的原因：

- 每元素 8 Byte；
- 参数、激活和带宽压力是 FP32 的 2 倍、BF16 的 4 倍；
- AI GPU 的 FP64 吞吐通常不是为大模型 Tensor Core 热路径设计的；
- Transformer 训练一般不需要如此高的精度。

但如果你做的是极其敏感的 `torch.linalg`、条件数很差的分解，FP64 可能是诊断工具。

3.2 FP32：稳定基线与“精度锚点”

FP32 长期是神经网络训练基线：

- 动态范围大；
- 约 7 位十进制有效数字；
- 数值调试相对容易；
- 适合作为累加器、优化器状态和敏感归约精度。

但它的代价是：

```
每个值 4 Byte
```

相对 BF16/FP16：

- 参数/激活原始存储翻倍；
- HBM 流量翻倍；
- 通信 payload 翻倍；
- 在支持低精度 Tensor Core 的 GPU 上，矩阵计算吞吐通常明显更低。

现代大模型很少“所有地方都 FP32”，而是让 FP32 留在最需要稳定性的地方。

3.3 TF32：FP32 接口下的 Tensor Core 快速矩阵模式

TF32 的正确心智模型：

```
存储：通常仍是FP32 Tensor
GEMM/Conv输入有效精度：约8-bit指数 + 10-bit尾数
乘加/输出：使用Tensor Core和FP32级累加路径
```

它保留 FP32 的指数范围，但有效尾数接近 FP16，因此：

- 比完整 IEEE FP32 GEMM 更快；
- 不需要把模型显式转成 `torch.float16`；
- **不会把参数显存从 4 Byte 降成 2 Byte**；
- 数值误差高于完整 FP32。

截至 PyTorch 2.13 的接口趋势：

```
torch.backends.cuda.matmul.fp32_precision = "tf32"
torch.backends.cudnn.conv.fp32_precision = "tf32"
```

需要完整 IEEE FP32 时：

```
torch.backends.cuda.matmul.fp32_precision = "ieee"
torch.backends.cudnn.conv.fp32_precision = "ieee"
```

TF32 适合：

- 想保留 FP32 Tensor API；
- 模型对 TF32 误差不敏感；
- 不急于节省参数/激活显存；
- 希望先以最小代码改动获得 Tensor Core 加速。

3.4 BF16：现代 LLM 训练最常见的低精度默认值

BF16 位布局：

```
1 Sign + 8 Exponent + 7 Mantissa
```

它的核心设计是：

```
牺牲尾数精度
换取接近FP32的动态范围
```

优点：

- 每元素 2 Byte；
- 指数范围接近 FP32；
- 大梯度、大激活不容易像 FP16 那样 Overflow；
- 小梯度也比 FP16 更不容易因指数范围而 Underflow；
- 大模型预训练通常不需要 Loss Scaling；
- Ampere 及以后 GPU 对 BF16 Tensor Core 支持成熟。

缺点：

- 只有 7 位显式尾数，`1.0` 附近间隔约为 `0.0078125`；
- 直接用 BF16 保存长期的小更新，可能被舍入掉；
- Reduction、Optimizer State、Softmax/Norm 等仍常需 FP32；
- 并非所有算子和硬件都同样高效。

一句话：

> BF16 选择“范围优先”，非常适合数值分布跨度大的深度学习训练。

3.5 FP16：尾数较细，但范围明显更窄

FP16 位布局：

```
1 Sign + 5 Exponent + 10 Mantissa
```

关键范围：

```
最大有限值：65504
最小正规正数：约6.10e-5
最小Subnormal：约5.96e-8
```

相对 BF16：

- FP16 在 `1.0` 附近更精细；
- 但指数范围窄很多；
- 大值容易 Overflow 为 Inf；
- 小梯度容易 Underflow 为 0。

因此 FP16 训练通常需要 Dynamic Loss Scaling。

适合：

- GPU 不支持 BF16、但支持 FP16 Tensor Core；
- 已验证模型在 FP16 下稳定；
- 需要较细的局部尾数精度；
- 有成熟 AMP/GradScaler 流程。

不适合直接把一个 BF16 预训练模型无脑转换成 FP16：该模型训练中形成的数值范围可能超过 FP16 的 `65504`。

3.6 FP8：不只是把 BF16 再砍一半

FP8 通常有两种互补格式：

```
E4M3：指数4位、尾数3位
      精度相对更好，范围更小，最大幅值约448

E5M2：指数5位、尾数2位
      精度更差，范围更大，最大幅值约57344
```

典型 Hybrid Recipe：

```
Forward Weight / Activation: E4M3
Backward Gradient: E5M2
Accumulator / Sensitive Ops: 更高精度
```

为什么 FP8 必须 Scaling?

不同层、不同张量的 `amax = max(abs(x))` 可能相差几个数量级。若直接转 FP8：

- Scale 太小：大值 Saturate/Overflow；
- Scale 太大：大多数小值量化成 0；
- 一个全局 Loss Scale 无法适配所有 Activation、Weight 和 Gradient。

概念上：

```
x_scaled = x × scale
x_fp8 = cast_to_fp8(x_scaled)
x_reconstructed ≈ cast_high(x_fp8) ÷ scale
```

常见 Scaling：

- **Delayed Scaling**：使用过去若干 Step 的 `amax history` 计算本 Step Scale；性能好，但对分布突变有滞后；
- **Current Scaling**：基于当前张量范围决定 Scale；更及时，但朴素实现可能需要额外遍历；
- **Block/Microscaling**：每 16/32 个值共享一个 Scale，降低 Outlier 对整张 Tensor 的影响。

FP8 的额外成本：

- 维护 Scale 与 `amax history`；
- Cast/Quantize/Dequantize；
- 可能需要 Scale 同步；
- Outlier、Saturation 和分布漂移监控；
- Shape/Kernel/硬件约束。

所以 FP8 是否更快取决于：

```
Tensor Core收益 + HBM/通信节省
>
量化、Scaling、同步和不支持算子的回退开销
```

3.7 INT8 与 INT4：量化，不是普通浮点缩窄

整数没有指数。要用 INT8/INT4 表示浮点张量，需要 Scale，可能还需要 Zero Point。

典型仿射量化：

```
q = clip(round(x ÷ scale) + zero_point)

x_reconstructed
≈ scale × (q − zero_point)
```

量化粒度：

- Per-Tensor：整张 Tensor 一个 Scale；
- Per-Channel：每个输出通道一个 Scale；
- Per-Group：每组 32/64/128 个权重一个 Scale；
- Per-Token：每个 Token/行一个 Scale；
- Per-Block：二维块共享 Scale。

INT8 常见用途：

- Weight + Activation 推理；
- PTQ/QAT；
- 部分低精度通信；
- KV Cache 量化。

INT4 常见用途：

- Weight-Only Quantization；
- LLM 推理中压缩模型权重；
- 让更大模型放入更少 GPU；
- 减少权重读取带宽。

但 INT4 推理可能从 GEMM Compute-Bound 变成：

```
反量化开销 + Scale读取 + 小Batch GEMV + Kernel效率
```

因此 4-bit 权重不保证端到端速度一定是 BF16 的 4 倍。

3.8 FP4 / NVFP4：前沿低精度训练，不是“裸 E2M1”

FP4 E2M1 原始可表示值非常稀疏，幅度约到 `±6`。单独使用几乎无法覆盖 LLM 的 Activation、Weight 和 Gradient 分布。

NVFP4 通过 Recipe 才变得可用：

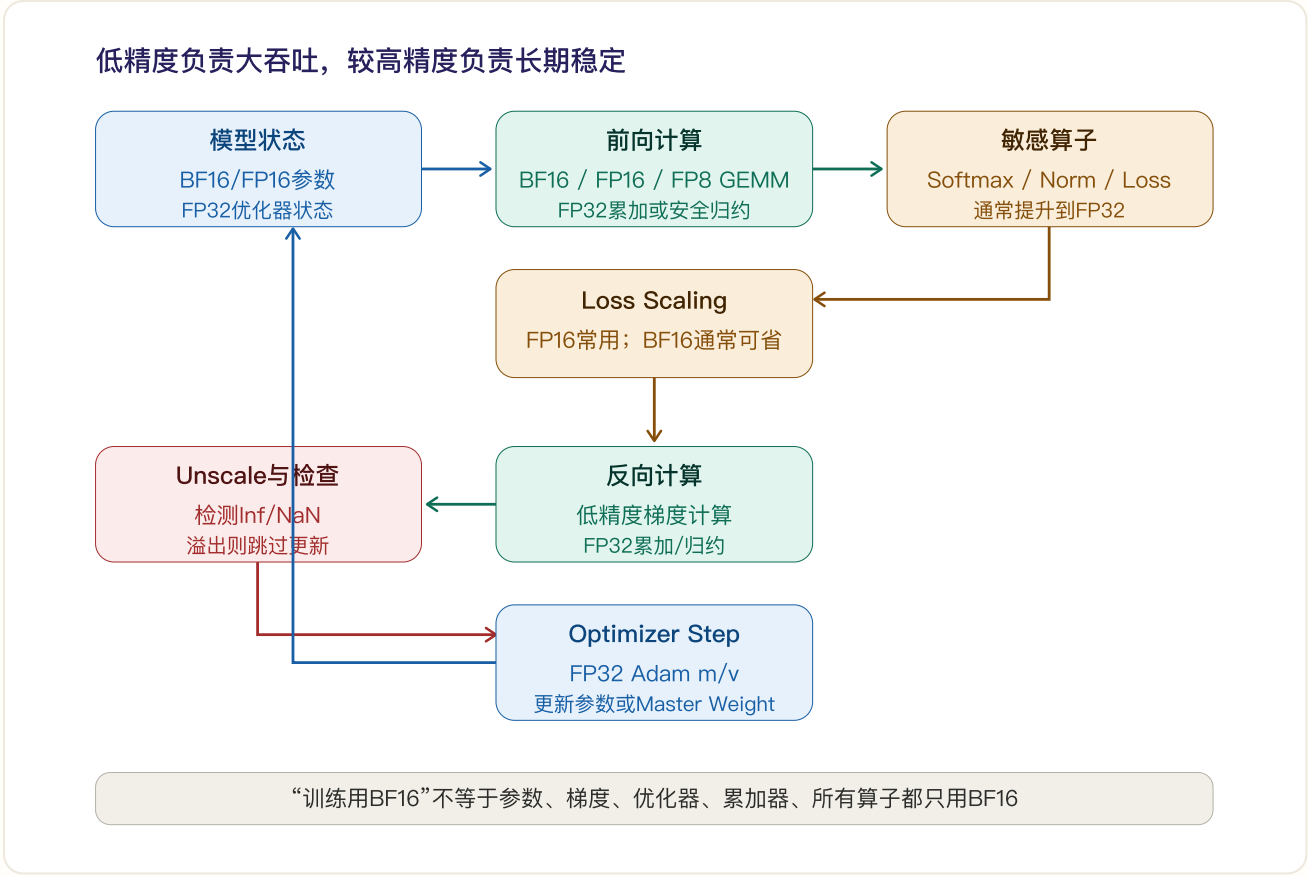
- 每 16 个值共享一个 E4M3 Block Scale；
- 额外使用 Per-Tensor FP32 Scale 防止全局 Overflow；
- Weight 可使用二维 16×16 Block Scaling；
- Gradient 使用 Stochastic Rounding 减少舍入偏差；
- 使用 Hadamard Transform 平滑 Outlier；
- 部分张量和敏感操作继续保留更高精度。

因此：

> “FP4 训练”不是所有状态都存成 4 bit，而是专用 Tensor Core、微块缩放、随机舍入、分布整形和混合精度共同组成的训练 Recipe。

截至 2026-08-02，这仍强依赖 Blackwell 级硬件与 Transformer Engine 等专用软件栈，不能把它当作任何 GPU 上的通用默认训练精度。

四、混合精度训练到底混合了什么



真实训练至少要区分以下精度：

对象/阶段	常见精度	为什么
GEMM输入权重	BF16/FP16/FP8	Tensor Core吞吐、减少HBM读取
激活	BF16/FP16，部分FP8	显存大户，低精度收益明显
GEMM乘积累加	FP32或硬件定义的较高精度	长Reduction易累积误差/溢出
Softmax输入与Reduction	通常提升FP32	Exp和求和数值敏感
LayerNorm/RMSNorm统计	常用FP32统计	均值/方差/平方和敏感
Loss	通常FP32	训练信号和归约稳定性
Gradient工作张量	BF16/FP16/FP8混合	省显存/通信，但需避免下溢/溢出
Gradient Accumulation Buffer	BF16或FP32，依实现	稳定性与显存取舍
Adam first/second moment	常见FP32	长期累计历史，需要精度与范围
Master Weight	FP32，是否存在依实现	保存小更新；某些BF16优化器布局可不单独保留
Checkpoint	BF16/FP16或混合状态	取决于恢复训练和部署需求
DP/TP/PP/EP通信	BF16/FP16/FP8等	降低通信量，但Collective精度需验证

4.1 “输入精度、累加精度、输出精度”必须分开

一个矩阵乘：

```
C = A × B
```

可能是：

```
A: BF16
B: BF16
乘法: BF16输入Tensor Core
Accumulator: FP32
C输出: BF16
```

也可能因性能开关，在长 Reduction 中允许部分 Reduced-Precision Reduction。PyTorch 为 FP16/BF16 GEMM 提供相关控制：

```
torch.backends.cuda.matmul.allow_fp16_reduced_precision_reduction = False
torch.backends.cuda.matmul.allow_bf16_reduced_precision_reduction = False
```

关闭可能提高稳定性，但也可能降低性能。不能只看 Tensor dtype 就断言内部每一步是什么精度。

五、FP16 为什么需要 Loss Scaling

设原始 Loss 为 `L`，参数为 `W`：

```
g = ∂L/∂W
```

若 `g` 很小，转成 FP16 可能变成 0。

把 Loss 乘以 Scale `S`：

```
L_scaled = L × S

g_scaled
= ∂(L×S)/∂W
= S × g
```

Backward 后再除回去：

```
g = g_scaled ÷ S
```

动态 Loss Scaling：

```
若Gradient出现Inf/NaN:
    跳过Optimizer Step
    降低Scale

若连续多个Step都有限:
    逐步提高Scale
```

PyTorch 典型写法：

```
import torch

model = model.cuda()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
scaler = torch.amp.GradScaler("cuda")

for x, target in dataloader:
    x = x.cuda(non_blocking=True)
    target = target.cuda(non_blocking=True)
    optimizer.zero_grad(set_to_none=True)

    with torch.autocast(
        device_type="cuda",
        dtype=torch.float16,
    ):
        output = model(x)
        loss = loss_fn(output, target)

    scaler.scale(loss).backward()

    # 若要做Gradient Clipping，先unscale
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(
        model.parameters(),
        max_norm=1.0,
    )

    scaler.step(optimizer)
    scaler.update()
```

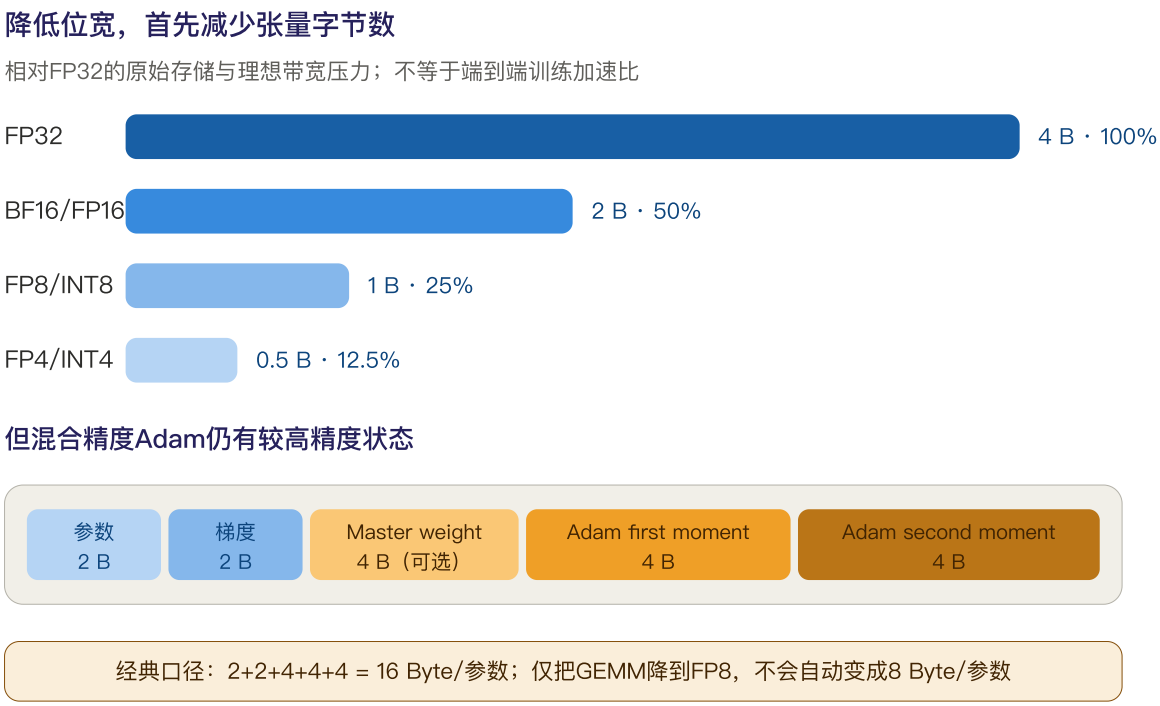
BF16 通常可以：

```
with torch.autocast(
    device_type="cuda",
    dtype=torch.bfloat16,
):
    output = model(x)
    loss = loss_fn(output, target)

loss.backward()
optimizer.step()
```

工程上仍应检查实际硬件、框架和模型；“BF16 通常不需要 GradScaler”不等于所有平台和自定义算子永远不需要任何稳定措施。

六、精度如何影响显存



6.1 只看模型权重：7B 参数

设：

```
P = 7×10^9 params
```

权重存储：

```
FP32: 7e9 × 4 Byte = 28 GB ≈ 26.08 GiB
BF16/FP16: 7e9 × 2 Byte = 14 GB ≈ 13.04 GiB
FP8/INT8: 7e9 × 1 Byte = 7 GB ≈ 6.52 GiB
INT4/FP4: 7e9 × 0.5 Byte = 3.5 GB ≈ 3.26 GiB
```

这是**权重本身**，不含：

- Scale/Zero Point；
- Padding/Alignment；
- KV Cache；
- Activation；
- CUDA Workspace；

- 框架元数据；
- 训练梯度和优化器状态。

6.2 训练状态：为什么 BF16 训练不是 2 Byte/参数

经典混合精度 Adam 口径：

```
低精度参数          2 Byte
低精度梯度          2 Byte
FP32 Master Weight  4 Byte
FP32 Adam m          4 Byte
FP32 Adam v          4 Byte
--------------------------
合计约              16 Byte/参数
```

7B：

```
7e9 × 16 Byte
= 112 GB
≈ 104.31 GiB
```

某些 BF16 训练布局不单独保留 FP32 Master Weight，则可能接近：

```
2 + 2 + 4 + 4 = 12 Byte/参数
```

但真实口径取决于：

- Optimizer 实现；
- 是否有 FP32 Parameter Copy；
- Gradient Accumulation dtype；
- ZeRO/FSDP 分片；
- 是否使用 8-bit Optimizer；
- Padding、Bucket、Flat Parameter 和临时 Buffer。

因此报告显存时必须说清“每参数包含哪些状态”。

6.3 Activation 示例

统一 Shape：

```
B = 2
S = 4096
```

```
H = 8192
Activation Shape = [B,S,H]
```

元素数：

```
N = 2 × 4096 × 8192
  = 67,108,864 elements
```

一个 Activation Tensor：

```
FP32: 67,108,864 × 4 B = 256 MiB
BF16/FP16: × 2 B = 128 MiB
FP8: × 1 B = 64 MiB
```

但完整训练 Activation 显存还乘以：

- 层数；
- 需要为 Backward 保存的中间张量；
- Microbatch 数；
- Attention/MLP 中间值；
- 是否 Activation Checkpoint；
- TP/SP/CP 分片；
- Fused Kernel 是否避免物化中间 Tensor。

七、精度如何影响 HBM 带宽与通信

7.1 HBM 带宽

一个 Memory-Bound Kernel 读取 `N` 个元素并写 `N` 个元素：

```
Bytes ≈ N × (read_count + write_count) × bytes_per_element
```

若从 FP32 降到 BF16：

```
每元素4 B → 2 B
理论流量减半
```

如果 Kernel 完全由 HBM 带宽限制、且没有额外 Cast/Scale，理论上时间可以接近减半。

但实际可能受限于：

- Kernel Launch；
- Index/Metadata；
- Cache 命中；
- 反量化；
- 不支持低精度的算子；
- 访存未合并；
- 小 Shape 无法打满带宽。

7.2 分布式通信

7B 梯度：

```
FP32 Gradient Payload = 28 GB
BF16/FP16 Payload = 14 GB
FP8 Payload = 7 GB（若通信与归约支持并且数值可接受）
```

Ring AllReduce 每 Rank 理想收发量：

```
Bytes_per_rank
≈ 2 × (N−1)/N × Payload
```

8 卡：

```
系数 = 2 × 7/8 = 1.75

FP32：1.75 × 28 GB = 49 GB/Rank
FP16：1.75 × 14 GB = 24.5 GB/Rank
```

这说明降低通信 dtype 可显著减少带宽压力。

但要注意：

- 梯度量化需要 Scale；
- AllReduce 是多次累加，误差会传播；
- Error Feedback、Chunk Scaling、FP32 Accumulation 可能需要额外 Buffer；
- 网络固定延迟不会因 payload 减半而消失；
- 通信与计算 Overlap 后，减少的可能是“隐藏时间”，不一定减少同等 Step Time。

八、精度如何影响计算性能

8.1 为什么低精度 Tensor Core 更快

更低位宽允许硬件在同样的：

- 晶体管预算；
- 数据通路宽度；
- 寄存器/共享内存带宽；
- Tensor Core 单元；

中并行处理更多元素。

因此在支持的硬件上，通常存在：

```
FP8 Tensor Core Throughput > BF16/FP16 > TF32 > IEEE FP32
```

但具体峰值倍数依 GPU 型号、稀疏模式、功耗和数据布局而变，不能脱离硬件规格给统一数字。

8.2 端到端速度为什么不会按位数同比增加

假设训练 Step 中：

```
70% 时间是可降精度GEMM
30% 时间是通信、归一化、调度、数据加载等
```

若 GEMM 加速 2 倍：

```
Speedup
= 1 ÷ (0.30 + 0.70 ÷ 2)
= 1 ÷ 0.65
≈ 1.54×
```

若 GEMM 加速 4 倍：

```
Speedup
= 1 ÷ (0.30 + 0.70 ÷ 4)
= 1 ÷ 0.475
≈ 2.11×
```

即使 GEMM 无限快，上限也只有：

```
Speedup_max = 1 ÷ 0.30 ≈ 3.33×
```

这是 Amdahl's Law。

低精度端到端收益取决于：

- GEMM 占比；
- Shape 是否满足 Tensor Core 对齐；
- GEMM 是否足够大；
- FP8/FP4 Scale 与 Cast 是否融合；
- HBM 或网络是否成为新瓶颈；
- CPU Launch/Data Loader 是否成为瓶颈；
- 是否因不稳定而回退或重跑。

九、精度如何影响模型质量与训练稳定性

9.1 四类主要数值风险

Overflow

值的绝对值超过最大有限值：

```
FP16中 |x| > 65504
→ Inf
```

后续常传播为 NaN。

Underflow

绝对值过小：

```
小Gradient → 0
```

该参数本 Step 不再获得有效更新。

Rounding

真实值被舍入到最近可表示值：

```
BF16：1.0 + 0.001
可能仍等于1.0
```

Accumulation Error

长 Reduction 的求和顺序、分块方式和 Accumulator dtype 都会影响结果：

```
(a + b) + c
不一定等于
a + (b + c)
```

这也是不同 Kernel、不同 Batch 实现、CPU/GPU 结果不保证 Bitwise Identical 的原因。

9.2 哪些算子更敏感

通常需要谨慎的区域：

- Softmax 的 `exp` 与归一化；
- Cross Entropy / LogSumExp；
- LayerNorm/RMSNorm Reduction；
- 大规模 Sum/Mean；
- Optimizer m/v 更新；
- 梯度裁剪范数；
- 概率极小或极大的路由；
- Ill-Conditioned Linear Algebra；
- 长序列 Attention 的在线归约。

AMP 的意义就是：

```
安全的大GEMM用低精度
敏感操作自动提升精度
```

十、训练、微调和推理分别怎么选

10.1 大模型预训练

优先顺序：

```
BF16：通用稳健默认
FP8：硬件/框架成熟时追求更高吞吐
FP16：BF16不可用或已验证稳定时
FP32/TF32：数值调试、敏感任务或基线
FP4：专用新硬件与成熟Recipe下评估
```

推荐实践：

- BF16 参数/激活；
- FP32 Optimizer State；
- FP32 Norm/Softmax/关键 Reduction；
- 观察 Loss、Grad Norm、Inf/NaN、Overflow、Amax；
- 低精度改变后必须做收敛对齐，而不只是吞吐 Benchmark。

10.2 Fine-Tuning

- 从 BF16 预训练 Checkpoint 继续训练，优先 BF16；
- LoRA/QLoRA 中 Base Weight 可为 INT4/NF4，Adapter/Optimizer 仍使用 BF16/FP32；
- 小数据微调对数值噪声和超参数可能更敏感；
- FP16 下重点观察 Overflow 与 GradScaler。

10.3 推理

常见路线：

```
BF16/FP16：质量稳、部署简单
FP8：权重+激活低精度，需硬件支持和校准/Scaling
INT8：成熟的Weight+Activation量化
INT4：常用于Weight-Only，显著压缩权重
FP4：新硬件的更激进低精度方案
```

推理要分别看：

- Prefill：大 GEMM，容易吃到 Tensor Core 收益；
- Decode：小 Batch/GEMV，常受权重带宽、KV Cache 和调度影响；
- KV Cache：精度降低可直接扩大并发和上下文，但需验证 Attention 质量；
- Logits/Softmax/Sampling：通常不应无脑降到最窄精度。

十一、一个可运行的 PyTorch 精度检查示例

```
import torch

for dtype in [
    torch.float32,
    torch.bfloat16,
    torch.float16,
]:
    info = torch.finfo(dtype)

    print("dtype:", dtype)
    print("bytes:", info.bits // 8)
    print("max:", info.max)
    print("smallest_normal:", info.smallest_normal)
    print("eps near 1:", info.eps)
    print()

# 观察大数吃小数
for dtype in [
    torch.float32,
    torch.float16,
    torch.bfloat16,
]: dtype=dtype)
    delta = torch.tensor(1e-4, dtype=dtype)

    print(
        dtype,
        "1 + 1e-4 =",
        (one + delta).item(),
    )
```

你应看到：

- FP32 可以保留 `1e-4` 级变化；
- FP16 在 `1.0` 附近的间隔约 `9.77e-4`；
- BF16 在 `1.0` 附近的间隔约 `7.81e-3`；
- 因此 FP16/BF16 都可能把 `1 + 1e-4` 舍入为 `1.0`。

但这不表示 BF16 不适合训练，因为大模型训练会使用：

- FP32 Accumulation；
- FP32 Optimizer State；

- 合适的学习率/更新路径；
- 混合精度算子策略。

十二、FP8 Transformer Engine 最小示意

以下示例表达 API 形态，具体 Recipe 和支持范围依 Transformer Engine/GPU 版本：

```
import torch
import transformer_engine.pytorch as te
from transformer_engine.common import recipe

layer = te.Linear(
    in_features=4096,
    out_features=16384,
    bias=False,
).cuda()

x = torch.randn(
    8,
    4096,
    device="cuda",
    dtype=torch.bfloat16,
)

fp8_recipe = recipe.DelayedScaling(
    fp8_format=recipe.Format.HYBRID,
    amax_history_len=16,
    amax_compute_algo="max",
)

with te.autocast(
    enabled=True,
    recipe=fp8_recipe,
):
    y = layer(x)
    loss = y.float().square().mean()

# backward通常在autocast作用域之外触发
loss.backward()

print(y.shape)
```

`Format.HYBRID` 的典型含义：

```
Forward: E4M3
Backward Gradient: E5M2
```

Transformer Engine 管理：

- FP8 Cast；
- Scale；
- Amax History；

- 支持的 FP8 Linear/Transformer Kernel；
- 必要的更高精度路径。

十三、如何选择精度：工程决策表

需求/约束	首选	原因	需要重点验证
现代LLM预训练稳健默认	BF16 Mixed Precision	FP32级范围、2Byte、通常无需Loss Scaling	收敛、FP32 Reduction、硬件吞吐
BF16硬件不可用	FP16 + GradScaler	Tensor Core成熟、2Byte	Overflow/Underflow、Scale变化、NaN
保持FP32存储且最小改动加速	TF32	FP32接口下加速GEMM/Conv	数值误差、PyTorch开关
Hopper/Blackwell追求训练吞吐	FP8 Recipe	1Byte输入、Tensor Core吞吐高	Amax、Saturation、Scale、回退算子
通用稳定基线/敏感归约	FP32	范围与精度较平衡	显存和吞吐成本
科学计算/病态线代	FP64	高精度	巨大性能/显存成本
推理兼顾质量与部署简单	BF16/FP16	无复杂量化或较少校准	显存容量、吞吐
推理压权重、扩大部署密度	INT4 Weight-Only	权重约0.5Byte/参数	反量化、Scale、质量、Kernel
推理Weight+Activation量化	INT8/FP8	降低权重和激活流量	Calibration、Outlier、算子覆盖
Blackwell前沿低精度训练	NVFP4	极低位宽+专用Recipe	Recipe成熟度、质量、硬件与软件版本

十四、低精度调优应该监控什么

数值稳定性

```
Loss Curve
Grad Norm
Inf/NaN Count
Dynamic Loss Scale
Skipped Optimizer Steps
Activation/Gradient Amax
FP8 Saturation Rate
Quantization Error
Per-Layer Output Error
```

性能

```
Step Time
Tokens/s
MFU
Tensor Core Utilization
GEMM Kernel Dtype
HBM Read/Write Throughput
Cast/Scale Kernel Time
Communication Exposed Time
Kernel Fallback Count
```

显存

```
Parameter Memory
Gradient Memory
Optimizer State
Master Weight
Activation Peak
KV Cache
Scale/Metadata
Communication Buffer
Workspace
Fragmentation
```

必须做两类对照：

1. **Numerical Parity**：相同数据/Seed 下，Loss、Grad、输出误差是否可接受；
2. **End-to-End Performance**：不是只看 GEMM Microbenchmark，而是看训练 Step 或 Serving 指标。

十五、常见误区

误区1：BF16 比 FP16“全面更精确”

错误。

```
BF16：范围更大，尾数更少
FP16：范围更小，尾数更多
```

BF16 更适合大模型训练，主要因为范围，而不是因为它每个局部数值都更精细。

误区2：模型用 BF16 训练，所有状态都是 BF16

错误。Optimizer State、Accumulation、Norm、Softmax、Loss 等经常保留 FP32。

误区3：FP8 相比 BF16 位数减半，显存一定再减半

只对真正以 FP8 常驻的张量成立。若参数本体、优化器、Checkpoint、部分激活仍为 BF16/FP32，总显存不会整体减半。

误区4：TF32 能把模型显存从 FP32 降一半

错误。TF32 主要是矩阵计算模式；Tensor 通常仍以 FP32 存储。

误区5：INT4 推理必然比 BF16 快4倍

错误。可能受反量化、Scale、GEMV、Kernel覆盖、Batch大小、KV Cache和调度限制。

误区6：累加器是FP32就绝对没有低精度误差

错误。输入量化已经丢失信息；Reduction 顺序、部分 Reduced-Precision Reduction、输出 Cast 仍会产生误差。

误区7：训练不出现NaN就说明精度安全

错误。还可能存在：

- 大量小梯度被舍入为0；
- 收敛速度变慢；
- 最终质量轻微下降；
- 某些层长期 Saturate；
- Router/Attention分布发生偏移。

十六、最终心智模型

```
指数位决定范围
尾数位决定精细程度
位宽决定存储、带宽和潜在计算密度

FP32：稳定锚点
TF32：FP32接口下的快速矩阵模式
BF16：范围优先的16位训练默认
FP16：精度较细但范围窄，需要Loss Scaling
FP8：需要逐张量/逐块Scaling的激进训练格式
INT8/INT4：主要通过量化服务推理
FP4：依赖微块Scaling与专用Recipe的前沿低精度
```

选精度时固定问六个问题：

1. **Storage**：参数、激活、梯度、KV Cache 分别以什么 dtype 常驻?
2. **Compute**：GEMM 输入、Accumulator、输出分别是什么精度?
3. **State**：Optimizer 与 Master Weight 是否保留 FP32?
4. **Scale**：是否需要 Loss Scaling、Per-Tensor Scale 或 Block Scale?
5. **Hardware**：GPU 是否原生支持该格式和高吞吐 Kernel?
6. **Quality**：是否完成训练收敛、下游指标和逐层误差对照?

真正的工程目标不是“使用最低位宽”，而是：

> **在目标质量不下降、训练不发散、运维可控的前提下，找到 Time-to-Train、显存、吞吐和成本最优的精度组合。**

可选自测

1. 格式辨析题

为什么 BF16 的局部尾数精度比 FP16 更低，却通常更适合 LLM 预训练？请从：

- 指数范围；
- Gradient Overflow/Underflow；
- Loss Scaling；

- FP32 Accumulation；

四个角度解释。

2. 显存计算题

一个 13B 模型：

```
参数与梯度：BF16
Master Weight：FP32
Adam m/v：FP32
```

请计算经典口径下参数相关训练状态的：

- Byte/参数；
- 总 GB；
- 总 GiB；
- 若 FSDP Full Shard 到 16 卡，理想每卡是多少？

不计 Activation、临时 AllGather 和 Workspace。

3. 工程决策题

某训练从 BF16 切到 FP8 后：

```
GEMM时间下降45%
Cast/Scale增加8% Step Time
AllReduce不变
某些层Amax偶发突增
最终Step Time只下降18%
```

请解释：

- 为什么端到端收益低于 GEMM 收益；
- Amax 突增可能带来什么风险；
- 应监控哪些指标；
- 你会尝试 Delayed Scaling、Current Scaling 还是更细粒度 Block Scaling，为什么？

继续学习

1. 前置：浮点舍入、Underflow/Overflow 与误差传播 —— 从数值分析解释训练为什么会发散。
2. 同层对比：BF16、FP16 与 FP8 混合精度训练源码路径 —— 深入 Autocast、GradScaler、Transformer Engine。
3. 下游应用：LLM INT8/INT4 量化 —— 深入 Per-Channel/Per-Group、GPTQ、AWQ、SmoothQuant 与 KV Cache 量化。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

截至 2026-08-02，本文稳定原理与接口主要核对：

- PyTorch 2.13 Numerical Accuracy；
- PyTorch AMP 官方文档；
- NVIDIA Mixed Precision Training Guide；
- NVIDIA Transformer Engine FP8/FP4 Primer 与用户指南；
- NVIDIA CUDA Programming Guide。

具体 Tensor Core 峰值、支持格式、Shape 约束、PyTorch 开关与 Transformer Engine Recipe 会随 GPU 和软件版本变化。生产环境应以实际 GPU 架构、CUDA/PyTorch/Transformer Engine 版本及 Profiler 结果为准。